

Plano de Refatoração para SOLID e Arquitetura Hexagonal

Diagnóstico técnico e melhorias justificadas, serviço a serviço

Sobre este documento

Este documento analisa o estado atual dos cinco microserviços do projeto (order, product, payment, inventory e orchestrator) e apresenta, de forma justificada, o conjunto de alterações necessárias para que o código atenda aos cinco princípios SOLID e ao estilo de Arquitetura Hexagonal (Ports & Adapters). Cada recomendação inclui o motivo da mudança, o impacto da violação atual e exemplos de código antes/depois.

Ficha técnica

Projeto	saga — SAGA orquestrada via Apache Kafka
Stack	Java 17, Spring Boot, Spring Kafka, JPA/PostgreSQL, MongoDB, Lombok
Serviços	order, product, payment, inventory, orchestrator
Autor	diogogape
Data	05/06/2026

Sumário

1. Sumário executivo
2. Visão geral da arquitetura atual
3. Diagnóstico SOLID (princípio a princípio)
4. Diagnóstico de Arquitetura Hexagonal
5. Estrutura de pacotes proposta
6. Refatoração detalhada (payment-service como exemplo)
7. Melhorias transversais (injeção, nomes, erros)
8. Resiliência e consistência de mensageria
9. Plano de migração incremental
10. Checklist de conformidade
11. Conclusão

Sumário executivo

O projeto implementa o padrão SAGA orquestrado: o order-service inicia a transação, e o orchestrator-service coordena as etapas de validação de produtos, pagamento e estoque, comunicando-se por tópicos Kafka. A lógica de negócio é funcional, porém a organização do código mistura regras de domínio com detalhes de infraestrutura (Kafka, JPA, Jackson, configuração do Spring), o que dificulta testes, evolução e troca de tecnologia.

Os problemas de maior impacto identificados são:

- **Inversão de dependência ausente:** as classes de serviço dependem diretamente de classes concretas de infraestrutura (KafkaProducer, JsonUtil, repositórios Spring Data, nomes de tópico via @Value), violando o DIP e impedindo testes isolados.
- **Sem portas nem adaptadores:** não existe separação entre domínio, aplicação e infraestrutura; o estilo é "camadas por tecnologia" (consumer/producer/service/dto), não Hexagonal.
- **Responsabilidades acumuladas:** métodos como PaymentService.realizePayment validam, persistem, mutam o evento, registram histórico e publicam no Kafka — tudo junto (violação do SRP).
- **Modelo de domínio confuso:** o objeto Event funciona ao mesmo tempo como modelo de domínio, DTO de transporte Kafka e portador de regras; entidades JPA carregam lógica.
- **Injeção por campo e falhas silenciosas:** uso de @Autowired em campos e tratamento de erro que engole exceções (JsonUtil retorna null/""; KafkaProducer apenas loga), gerando perda de mensagem e NPE.

Nenhum desses pontos exige reescrita: o documento propõe uma migração incremental e de baixo risco (Seção 9), começando por mudanças locais (injeção por construtor, nomes) e avançando até a separação completa em domínio / aplicação / infraestrutura.

Visão geral da arquitetura atual

2.1 Fluxo da SAGA

O fluxo de negócio percorre os serviços na seguinte ordem, coordenado pelo orquestrador, que decide o próximo tópico com base na origem e no status do evento (classe `SagaExecutionController`):

```
order-service      -> inicia a saga (start-saga)
orchestrator       -> PRODUCT_VALIDATION_SUCCESS
product-service    -> valida produtos          -> orchestrator
orchestrator       -> PAYMENT_SUCCESS
payment-service    -> realiza pagamento        -> orchestrator
orchestrator       -> INVENTORY_SUCCESS
inventory-service  -> baixa estoque            -> orchestrator
orchestrator       -> FINISH_SUCCESS / NOTIFY_ENDING
```

Em qualquer falha (`ROLLBACK_PENDING` / `FAIL`) o orquestrador dispara as compensações na ordem inversa (`rollback`).

2.2 Organização atual dos pacotes

Cada serviço repete a mesma estrutura, organizada por tipo técnico de componente e não por domínio:

```
com.saga.estudo.<servico>.service
|-- config/exception (ExceptionHandler, ValidationException)
|-- config/kafka     (KafkaConfig)
|-- consumer         (KafkaConsumer)      <- entrada
|-- producer         (KafkaProducer)      <- saida
|-- dto / document   (Event, Order, History, Product...)
|-- model             (entidades JPA: Payment, Inventory...)
|-- repository       (Spring Data)
|-- service           (regras + orquestracao + infra) <- tudo aqui
`-- utils             (JsonUtil)
```

O problema não é a existência desses pacotes, e sim o sentido das dependências: o pacote `service` (que deveria conter o núcleo de negócio) importa e depende de `consumer/producer/utils/repository`, ou seja, o núcleo depende da casca. A Arquitetura Hexagonal inverte exatamente essa relação.

SEÇÃO 3

Diagnóstico SOLID

A seguir, cada princípio é avaliado com a violação concreta encontrada no código, o impacto e a correção proposta.

3.1 SRP — Princípio da Responsabilidade Única

Uma classe deve ter um, e apenas um, motivo para mudar.

VIOLAÇÃO

`PaymentService.realizePayment` concentra cinco responsabilidades distintas: validar o evento, aplicar regra de negócio do pagamento, persistir no banco, registrar histórico no próprio evento e publicar a mensagem no Kafka. O mesmo padrão se repete em `ProductService`, `InventoryService` e `OrchestratorService`.

```
public void realizePayment(Event event){
    try {
        event.validate();           // validacao
        validateExistPayment(event); // regra/persistencia
        createPendingPayment(event); // persistencia
        Payment payment = findById... // persistencia
        validatePayment(payment);    // regra de negocio
        changePaymentToSuccess(payment); // persistencia
        handleSuccess(event);        // muta evento + historico
    } catch (Exception e) {
        handleFailCurrentPayment(event, e.getMessage());
    }
    kafkaProducer.sendEvent(orchestratorTopic, ...); // mensageria
}
```

CORREÇÃO

- A regra de negócio (cálculo, validação de valor mínimo, transição de status do pagamento) pertence ao modelo de domínio (`Payment` / `Order`).
- A coordenação do caso de uso (sequência: valida !' persiste !' publica) pertence a um serviço de aplicação.
- A persistência e a publicação no Kafka pertencem a adaptadores de saída, atrás de portas (interfaces).
- Assim, cada classe passa a ter um único motivo para mudar: muda a regra !' muda o domínio; muda o banco !' muda o adaptador JPA; muda o broker !' muda o adaptador de mensageria.

3.2 OCP — Princípio Aberto/Fechado

Entidades de software devem estar abertas para extensão e fechadas para modificação.

VIOLAÇÃO

- O bloco `try/catch + sendEvent(orchestratorTopic, ...)` é copiado em todos os serviços de etapa; adicionar uma nova etapa da saga obriga a editar e duplicar essa estrutura.
- Os métodos `handleSuccess` / `handleFailCurrent...` / `addHistory` são reescritos quase idênticos em cada serviço — qualquer mudança no formato do histórico exige tocar em 4 classes.

CORREÇÃO

- Extrair um contrato de etapa da saga (ex.: SagaStep / use case) e centralizar o fluxo "executa regra !" registra histórico !" publica resultado" em uma classe-modelo (Template Method) ou em um componente de aplicação reutilizável.
- A montagem do histórico e a transição de status do evento devem viver no próprio Event (domínio), expostos como métodos de intenção (event.markSuccess(source), event.markRollbackPending(source, msg)), evitando repetição e centralizando a regra.

3.3 LSP — Princípio da Substituição de Liskov

Hoje praticamente não há herança/polimorfismo, então não há violação ativa — mas também não há abstrações que permitam substituição. Ao introduzir as portas (Seção 4), o princípio passa a ser relevante:

- Toda implementação de uma porta de saída (ex.: PaymentRepositoryPort) deve honrar o contrato sem efeitos surpresa: um adaptador em memória usado em testes deve ser substituível pelo adaptador JPA sem quebrar o serviço de aplicação.
- Evitar exceções inesperadas ou retornos nulos não previstos no contrato — por isso a Seção 7 recomenda substituir os retornos null de JsonUtil por contratos explícitos (Optional/exceção).

3.4 ISP — Princípio da Segregação de Interface

VIOLAÇÃO

Os consumidores Kafka dependem da classe de serviço concreta inteira (ex.: o KafkaConsumer de pagamento depende de PaymentService, que expõe realizePayment e rollbackPayment). Não há interfaces; cada adaptador enxerga mais do que precisa.

CORREÇÃO

- Definir portas de entrada estreitas e específicas por caso de uso: ProcessPaymentUseCase e RollbackPaymentUseCase.
- O listener de pagamento bem-sucedido depende apenas de ProcessPaymentUseCase; o de falha, apenas de RollbackPaymentUseCase. Cada cliente depende somente do método que utiliza.

3.5 DIP — Princípio da Inversão de Dependência

Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações. Esta é a violação central do projeto.

VIOLAÇÃO

- O serviço (alto nível) depende diretamente de classes concretas de baixo nível: KafkaProducer, JsonUtil, PaymentRepository (Spring Data/JPA) e do nome do tópico via @Value.
- Há injeção por campo com @Autowired em payment/product/inventory (order-service já usa construtor) — dependências implícitas, difíceis de testar e de tornar imutáveis.

```
@Service
public class PaymentService {
    @Value("${kafka.topic.orchestrator}") private String topic;
    @Autowired private JsonUtil jsonUtil;           // concreto
    @Autowired private KafkaProducer kafkaProducer; // concreto
    @Autowired private PaymentRepository repo;       // JPA/Spring
}
```

CORREÇÃO

- O domínio/aplicação define interfaces (portas); a infraestrutura as implementa. O serviço passa a depender de `PaymentRepositoryPort` e `SagaEventPublisherPort`, nunca de Kafka/JPA.
- Usar injeção por construtor (final), tornando as dependências explícitas e a classe testável sem o contêiner Spring.
- Remover o nome do tópico do serviço: a escolha do tópico é detalhe do adaptador de mensageria, não do caso de uso.

Diagnóstico de Arquitetura Hexagonal

4.1 O conceito (Ports & Adapters)

Na Arquitetura Hexagonal o núcleo da aplicação (domínio + casos de uso) fica isolado no centro e não conhece tecnologia. Toda comunicação com o mundo externo passa por portas (interfaces) implementadas por adaptadores:

- **Portas de entrada (in):** contratos dos casos de uso, chamados pelos adaptadores de entrada (controllers REST, consumidores Kafka).
- **Portas de saída (out):** contratos de que o núcleo precisa (persistir, publicar evento), implementados pelos adaptadores de saída (JPA, produtor Kafka).
- **Regra de ouro:** as dependências apontam sempre para dentro. O domínio não importa Spring, Kafka, JPA ou Jackson.

4.2 Violações atuais

- **Domínio acoplado a frameworks:** as classes de serviço importam `org.springframework.kafka`, repositórios JPA e o `JsonUtil` (Jackson). O "núcleo" não é independente.
- **Modelo único sobrecarregado:** Event é simultaneamente modelo de domínio, DTO serializado para o Kafka e objeto mutável compartilhado entre etapas (`setStatus/setSource` alteram estado global do fluxo).
- **Regra em entidade de persistência:** a entidade JPA `Payment` e os DTOs `Order/Event` carregam lógica (`calculateAmount`, `validate`), misturando modelo de persistência/transporte com regra de negócio.
- **Configuração no núcleo:** o nome do tópico (`@Value`) e o `ObjectMapper` vivem dentro do serviço — detalhes de infraestrutura no centro.

4.3 Camadas-alvo

```
[ Adaptadores de ENTRADA ]      (infra)
REST Controller / Kafka Consumer
      | chamam
      v
[ Portas de ENTRADA ]  (interfaces de caso de uso)
      |
[ APLICACAO ]  servico que orchestra o caso de uso
      | usa
[ Portas de SAIDA ]    (interfaces: repositorio, publisher)
      ^ implementadas por
[ Adaptadores de SAIDA ]      (infra)
JPA Repository / Kafka Producer / Serializador

[ DOMINIO ]  modelos e regras puras, no centro, sem imports de infra
```

SEÇÃO 5

Estrutura de pacotes proposta

Estrutura sugerida por serviço (exemplo do payment-service). A mesma ideia vale para os demais; o orchestrator não tem persistência de domínio, mas mantém domínio/aplicação/adaptadores.

```
com.saga.payment
|-- domain
|   |-- model          Payment, Order, Event, History (POJOs puros)
|   |-- exception      ValidationException (sem Spring)
|-- application
|   |-- port
|   |   |-- in         ProcessPaymentUseCase, RollbackPaymentUseCase
|   |   |-- out        PaymentRepositoryPort, SagaEventPublisherPort
|   |-- service        PaymentApplicationService (implementa os use cases)
|-- infrastructure
|   |-- adapter
|   |   |-- in
|   |   |   |-- messaging PaymentKafkaConsumer
|   |   |-- out
|   |       |-- persistence PaymentJpaEntity, PaymentJpaRepository,
|   |       |               PaymentPersistenceAdapter
|   |       |-- messaging   KafkaEventPublisherAdapter, EventMessage
|-- config              KafkaConfig, beans, mapeadores
```

Note a inversão: application define as portas (in e out); infrastructure as implementa e depende de application/domain — nunca o contrário.

Refatoração detalhada — payment-service

Aplicação completa do padrão a um serviço, servindo de molde para os demais.

6.1 Portas de entrada (casos de uso)

Interfaces estreitas (ISP) que descrevem o que o serviço faz, sem expor como faz:

```
package com.saga.payment.application.port.in;

public interface ProcessPaymentUseCase {
    Event process(Event event);
}

public interface RollbackPaymentUseCase {
    Event rollback(Event event);
}
```

Por quê: o adaptador de entrada (consumer) passa a depender de uma abstração de caso de uso (DIP/ISP). Permite trocar a implementação ou testar o listener com um dublê.

6.2 Portas de saída (o que o núcleo precisa)

```
package com.saga.payment.application.port.out;

public interface PaymentRepositoryPort {
    boolean existsByOrderAndTransaction(String orderId, String txId);
    Optional<Payment> findByOrderAndTransaction(String o, String t);
    Payment save(Payment payment);
}

public interface SagaEventPublisherPort {
    void publishToOrchestrator(Event event);
}
```

Por quê: o núcleo declara sua necessidade (persistir, publicar) em termos próprios. Some a dependência de Kafka, JPA e do nome do tópico dentro da regra. Em testes, basta uma implementação em memória.

6.3 Domínio com regra encapsulada

As transições de estado e o histórico saem do serviço e passam a ser intenções explícitas do modelo Event (corrige SRP e OCP):

```

package com.saga.payment.domain.model;

public class Event {
    // ... campos ...
    public void markSuccess(String source, String msg) {
        this.source = source;
        this.status = ESagaStatus.SUCCESS;
        addToHistory(source, status, msg);
    }
    public void markRollbackPending(String source, String msg) {
        this.source = source;
        this.status = ESagaStatus.ROLLBACK_PENDING;
        addToHistory(source, status, msg);
    }
    // addToHistory(...) cria History internamente
}

```

6.4 Serviço de aplicação (orquestra o caso de uso)

Depende somente de portas, por construtor, sem Kafka/JPA/Jackson e sem @Value de tópico:

```

package com.saga.payment.application.service;

@Service
@RequiredArgsConstructor
public class PaymentApplicationService
    implements ProcessPaymentUseCase, RollbackPaymentUseCase {

    private static final String SOURCE = "PAYMENT_SERVICE";
    private final PaymentRepositoryPort repository; // porta
    private final SagaEventPublisherPort publisher; // porta

    @Override
    public Event process(Event event) {
        try {
            event.validate();
            ensureNotProcessed(event);
            Payment payment = Payment.pendingFrom(event); // regra no dominio
            payment.confirm(); // valida valor min.
            repository.save(payment);
            event.markSuccess(SOURCE, "Payment successfully");
        } catch (ValidationException e) {
            event.markRollbackPending(SOURCE, e.getMessage());
        }
        publisher.publishToOrchestrator(event);
        return event;
    }
    // rollback(...) analogo
}

```

Por quê: testável sem Spring; a sequência do caso de uso fica explícita e única (SRP); a regra de valor mínimo migrou para Payment.confirm() (domínio).

6.5 Adaptador de saída — persistência (JPA)

A entidade JPA fica separada do modelo de domínio; o adaptador traduz entre os dois e implementa a porta:

```

package com.saga.payment.infrastructure.adapter.out.persistence;

@Repository
@RequiredArgsConstructor
public class PaymentPersistenceAdapter implements PaymentRepositoryPort {

    private final PaymentJpaRepository jpa; // Spring Data

    @Override public Payment save(Payment payment) {
        PaymentJpaEntity saved = jpa.save(PaymentJpaEntity.from(payment));
        return saved.toDomain();
    }
    // existsBy... / findBy... delegam ao jpa e mapeiam
}

```

Por quê: troca de banco ou de mapeamento não toca no domínio; a regra não fica refém das anotações @Entity. O modelo de persistência (PaymentJpaEntity) e o de domínio (Payment) evoluem independentemente.

6.6 Adaptadores de mensageria (entrada e saída)

```

// SAIDA: conhece o topico, o serializador e o broker
@Component
@RequiredArgsConstructor
public class KafkaEventPublisherAdapter
    implements SagaEventPublisherPort {
    private final KafkaTemplate<String,String> template;
    private final JsonSerializer serializer;
    @Value("${kafka.topic.orchestrator}") private String topic;
    @Override public void publishToOrchestrator(Event event) {
        template.send(topic, serializer.toJson(EventMessage.from(event)));
    }
}

// ENTRADA: depende apenas dos casos de uso (ISP)
@Component
@RequiredArgsConstructor
public class PaymentKafkaConsumer {
    private final ProcessPaymentUseCase process;
    private final RollbackPaymentUseCase rollback;
    private final JsonSerializer serializer;
    @KafkaListener(topics = "${kafka.topic.payment-success}")
    public void onSuccess(String payload) {
        process.process(serializer.toEvent(payload));
    }
}

```

Por quê: o nome do tópico e o Kafka ficam confinados ao adaptador (DIP); o consumer depende só do caso de uso de que precisa (ISP).

6.7 Separar modelo de domínio do DTO de transporte

Hoje o mesmo Event é mutado em sequência e serializado direto no Kafka. Recomenda-se um EventMessage (DTO de transporte) com mapeamento explícito de/para o Event de domínio. Isso evita que mudanças no formato do tópico vazem para a regra, e elimina o estado mutável compartilhado como meio de comunicação entre etapas.

Melhorias transversais

7.1 Injeção por construtor (consistência)

order-service usa @AllArgsConstructor; payment/product/inventory usam @Autowired em campos. Padronizar tudo em injeção por construtor (final + @RequiredArgsConstructor):

- Torna as dependências explícitas e obrigatórias (não há objeto meio-construído).
- Permite campos final (imutabilidade) e instanciação direta em testes, sem o contêiner.
- Evita dependências ocultas e ciclos silenciosos.

7.2 Tratamento de erros e contratos honestos

VIOLAÇÃO

```
public Event toEvent(String json) {
    try { return objectMapper.readValue(json, Event.class); }
    catch (Exception ex) { return null; }    // falha silenciosa -> NPE
}
public String toJson(Object o) {
    try { return objectMapper.writeValueAsString(o); }
    catch (Exception ex) { return ""; }    // publica string vazia
}
```

- Engolir a exceção e retornar null faz o erro reaparecer adiante como NullPointerException, longe da causa.
- KafkaProducer também apenas loga o erro de envio: a mensagem é perdida sem que ninguém saiba (saga trava).

CORREÇÃO

- Falhar de forma explícita: lançar uma exceção de serialização dedicada em vez de retornar null/"".
- Capturar apenas exceções de negócio esperadas (ValidationException) no caso de uso; deixar erros de infraestrutura propagarem para a política de erro do Kafka.
- Adotar idempotência (já há checagens existsBy...) e considerar DLQ/retry e o padrão Transactional Outbox (Seção 8).

7.3 Padronização de nomes (legibilidade)

Corrigir erros de digitação que prejudicam a leitura e a manutenção:

```
createorder      -> createOrder
calculateAmont   -> calculateAmount
calculeteTotalItems -> calculateTotalItems
rollbackInvetory -> rollbackInventory
statSagaConsumer -> startSagaConsumer
TRANSACTION_IS_FORMAT -> TRANSACTION_ID_FORMAT
findByOrderIdAnsTrans. -> findByOrderIdAndTransactionId
```

7.4 Regra fora de DTO e de entidade JPA

- Mover `calculateAmount/calculateTotalItems/validate` dos DTOs `Order/Event` para o domínio (objetos de valor ou métodos de domínio), mantendo os DTOs de transporte anêmicos e estáveis.
- Tirar regra da entidade `@Entity Payment`: a entidade é detalhe de persistência; a regra (status inicial, transições) vive no domínio.

Resiliência e consistência de mensageria

Estas melhorias reforçam a confiabilidade da saga e complementam a separação em portas/adaptadores:

- **Transactional Outbox:** gravar o evento de saída na mesma transação do banco (tabela outbox) e publicá-lo de forma assíncrona, garantindo atomicidade entre "persistir estado" e "publicar evento" — hoje, se o save funciona e o send falha, a saga para inconsistente.
- **DLQ e retry:** configurar política de repetição e dead-letter topic no consumidor, em vez de perder mensagens silenciosamente no catch do produtor.
- **Consumo idempotente:** as verificações `existsByOrderIdAndTransactionId` já apontam nessa direção; formalizar como garantia de idempotência por `(orderId, transactionId)`.
- **Chave de partição:** publicar com chave = `orderId/transactionId` para preservar ordem por pedido (hoje o envio usa apenas valor).

Plano de migração incremental

Ordem sugerida, do menor para o maior risco, mantendo o sistema funcional a cada passo:

- **Passo 1** — padronizar injeção por construtor e corrigir nomes (mudanças locais, sem alterar comportamento).
- **Passo 2** — extrair portas de saída (SagaEventPublisherPort, *RepositoryPort) e fazer KafkaProducer/repositórios as implementarem; o serviço passa a depender das interfaces.
- **Passo 3** — extrair portas de entrada (use cases) e fazer os consumidores dependerem delas.
- **Passo 4** — separar modelo de domínio do DTO de transporte (EventMessage) e da entidade JPA, com mapeadores.
- **Passo 5** — mover regra de DTO/entidade para o domínio (markSuccess, confirm, cálculos).
- **Passo 6** — reorganizar pacotes em domain / application / infrastructure.
- **Passo 7** — adicionar testes de unidade no domínio e na aplicação (agora possíveis sem Spring/Kafka/DB).
- **Passo 8** — adicionar resiliência: Outbox, DLQ/retry, idempotência formal.

Checklist de conformidade

SOLID

- ☐ SRP: regra no domínio, caso de uso na aplicação, I/O nos adaptadores.
- ☐ OCP: fluxo de etapa centralizado; histórico/transição no Event.
- ☐ LSP: portas com contratos honrados por todas as implementações.
- ☐ ISP: portas de entrada estreitas por caso de uso.
- ☐ DIP: serviço depende de portas; injeção por construtor; sem @Value de tópico no núcleo.

Arquitetura Hexagonal

- ☐ Domínio sem imports de Spring/Kafka/JPA/Jackson.
- ☐ Portas de entrada e de saída definidas na camada de aplicação.
- ☐ Adaptadores de entrada (REST/Kafka) e de saída (JPA/Kafka) isolados em infrastructure.
- ☐ Modelo de domínio separado do DTO de transporte e da entidade JPA.
- ☐ Configuração (tópicos, ObjectMapper) confinada à infraestrutura.

Qualidade e resiliência

- ☐ Sem retornos null/"" silenciosos; erros explícitos.
- ☐ Idempotência por (orderId, transactionId).
- ☐ Outbox / DLQ / retry para envio confiável.
- ☐ Nomes corrigidos e consistentes.
- ☐ Testes de unidade no núcleo sem contêiner.

Conclusão

O projeto já implementa corretamente a mecânica da SAGA orquestrada; o ponto a evoluir é arquitetural. A causa raiz da maioria das violações SOLID é a mesma: o núcleo de negócio depende de detalhes de infraestrutura. Ao introduzir portas e adaptadores e inverter essas dependências, resolvem-se simultaneamente o DIP, o SRP (cada camada com uma responsabilidade), o ISP (interfaces estreitas), o OCP (extensão sem edição) e abre-se caminho para testes reais do domínio.

A migração proposta é incremental e reversível: cada passo entrega valor isolado e mantém o sistema operante, permitindo adotar a Arquitetura Hexagonal sem um "big bang". O resultado é um código mais testável, mais fácil de evoluir e independente das tecnologias de mensageria e persistência atualmente empregadas.